

The Art of Debugging

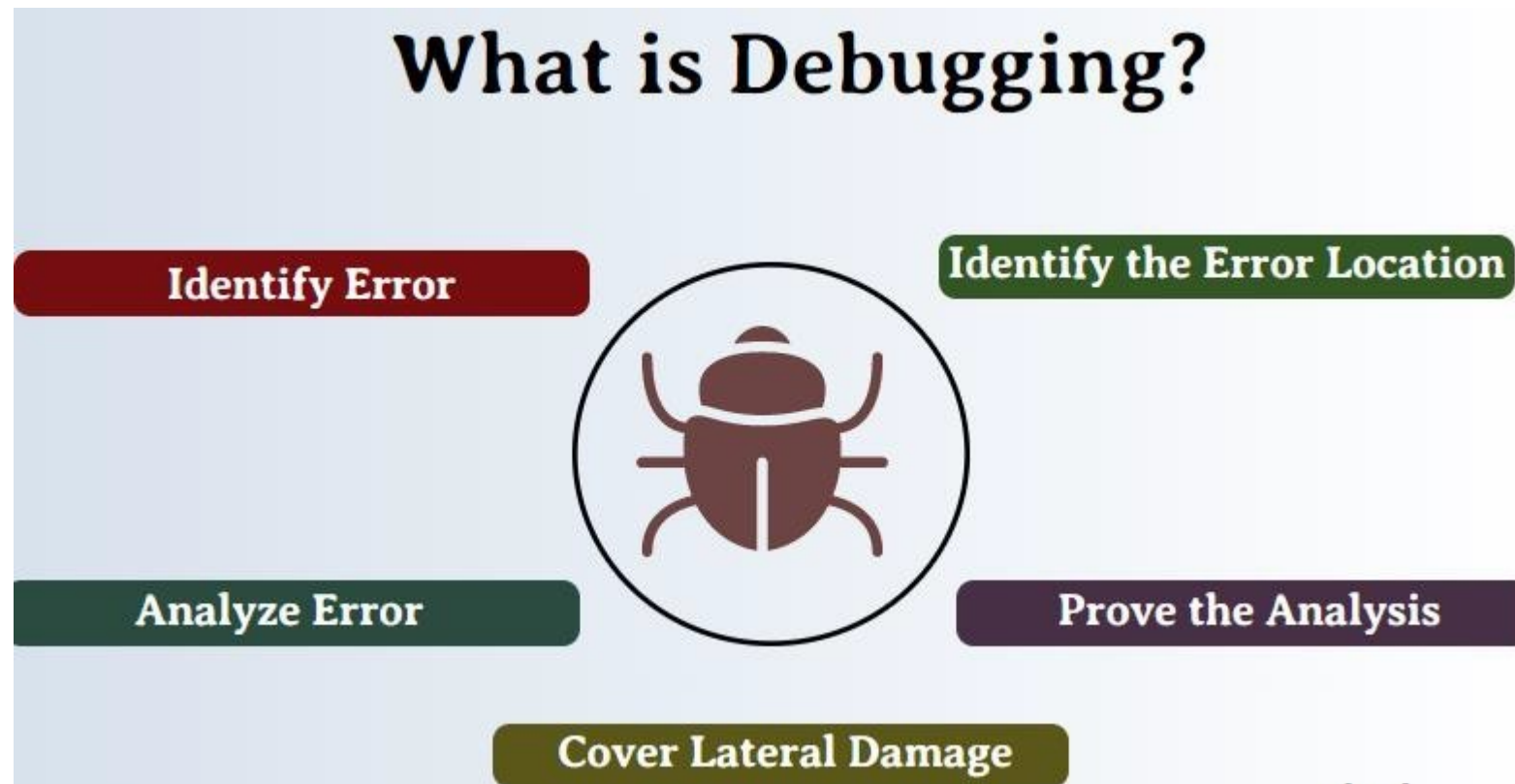
Execute



Debugging: A Diagnostic Process

The Debugging Process

- Debugging is the process that results in the **removal of the error**.
- Debugging occurs as a **consequence of successful testing**.

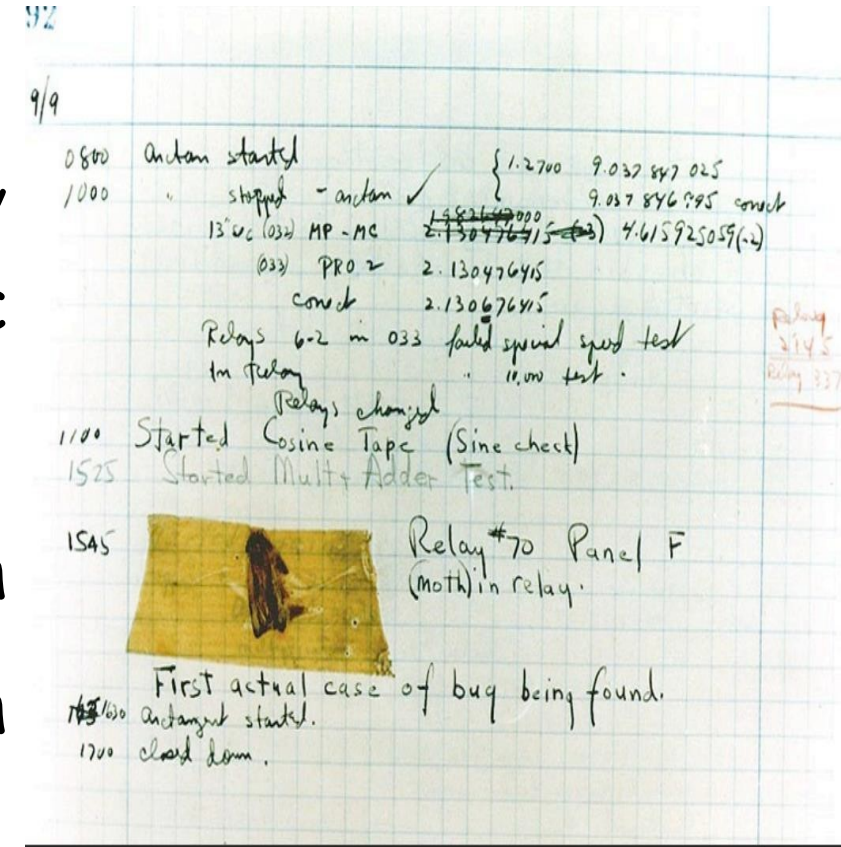


Origin of “debugging”



The “first computer bug” - Grace Hopper, working on one of the first electronic computers (Harvard Mark II, 1945)

-Machine failed and operator found a moth caught in a relay -Taped into log book with note: “First actual case of bug found”



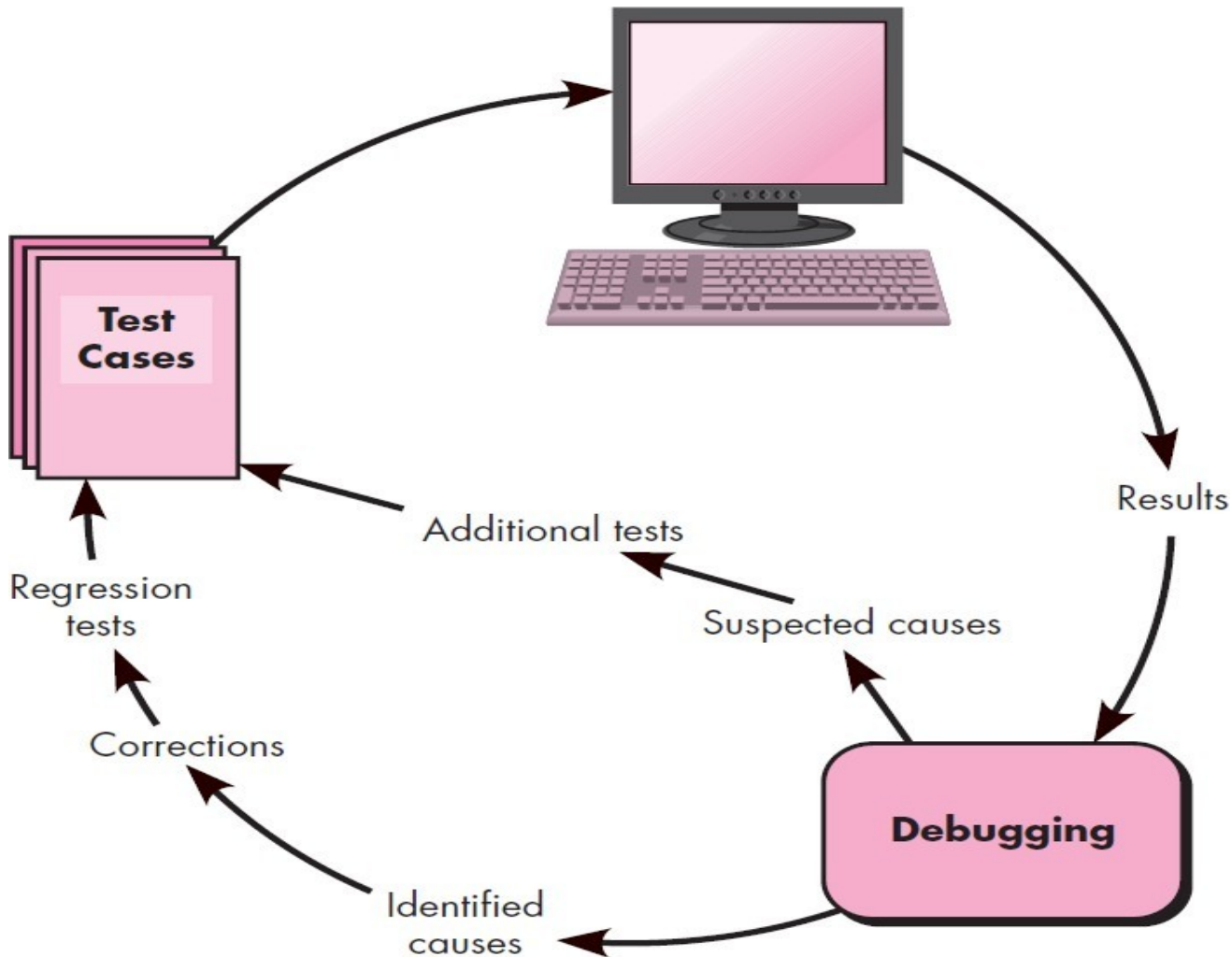
The Debugging Process Cont'd

The debugging process will usually have one of two outcomes:

- The cause will be found and corrected or
- The cause will not be found. In the latter case, the person performing debugging may suspect a cause, design a test case to help validate that suspicion and work toward error correction in an iterative fashion.

“if debugging is defined as the art of taking bugs out of a program, programming must be the art of putting them in -Dijkstra

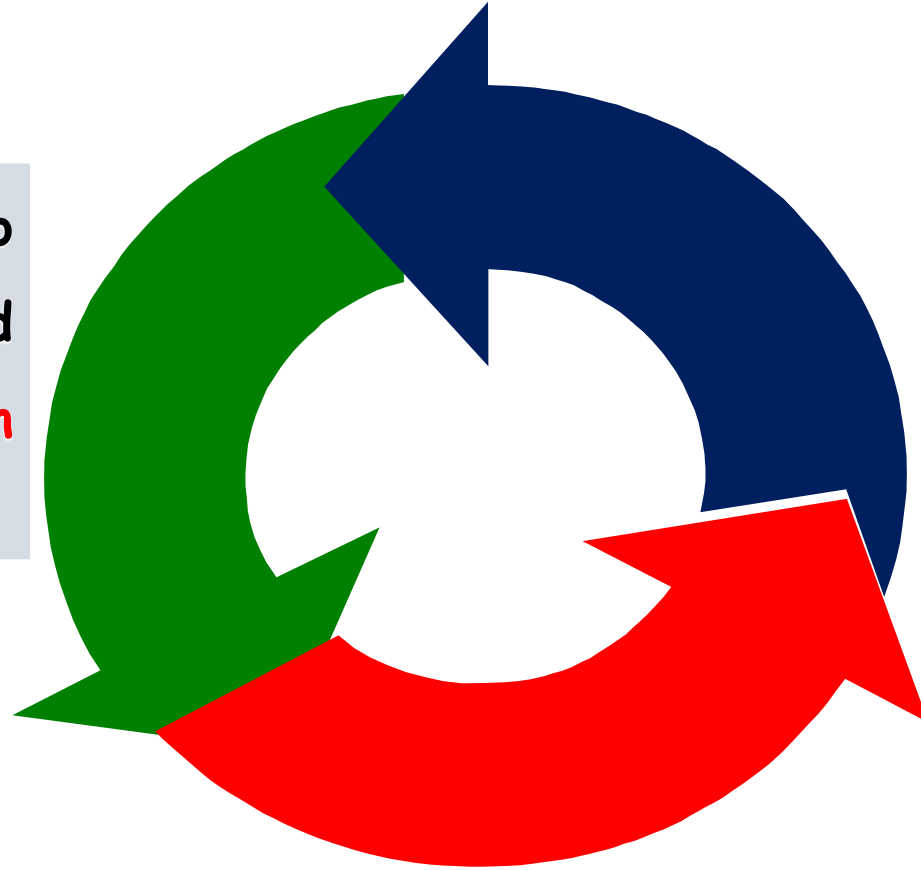
The Debugging Process Cont'd



- Debugging and testing are **complementary** processes.
- The purpose of **testing** is to identify what happens when there is a **mistake in a program's source code**.
- The purpose of debugging is to **locate and fix the mistake**.

Debugging Effort

Time required to
correct the error and
conduct **regression**
tests



Time required to **diagnose**
the symptom and **determine**
the cause

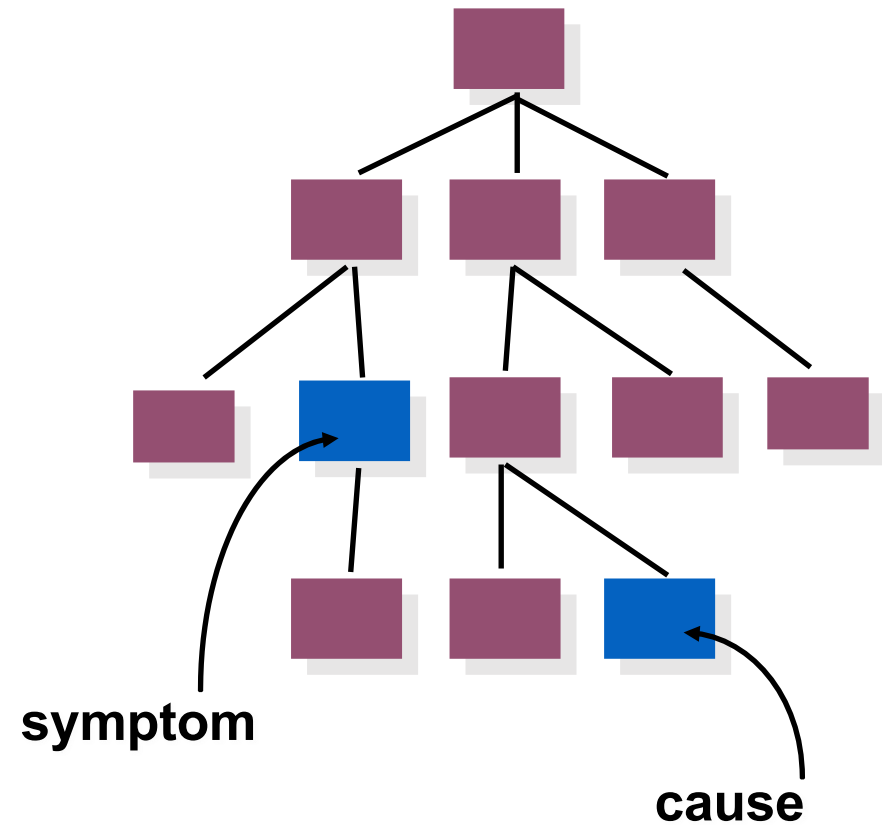
Why is debugging difficult?

- Debugging is one of the more **frustrating parts** of programming. It has elements of problem-solving or **brain teasers, coupled** with the annoying **recognition** that you have made a mistake.

- Cause and effect may be **hard to connect** (remote in time/space)
- **Symptoms** may seem **random** (the result of 2 bugs interacting)
- Complex interactions
- Psychological issues: **frustration, pressure, guilt**

Symptoms & Causes

- The symptom and the cause may be **geographically remote**
- The symptom may **disappear** (temporarily) when **another error is corrected**
- The symptom may actually be caused by **nonerrors**
- The symptom may be caused by **human error** that is **not easily traced**.
- The symptom may be **intermittent**
- The symptom may be a **result of timing problems**



Debugging Strategies

Debugging Strategies

```
graph LR; A[Debugging Strategies] --> B[Brute force]; A --> C[backtracking]; A --> D[induction]; A --> E[deduction];
```

Brute force

- The brute force category of debugging is probably the **most common** and **least efficient method**

backtracking

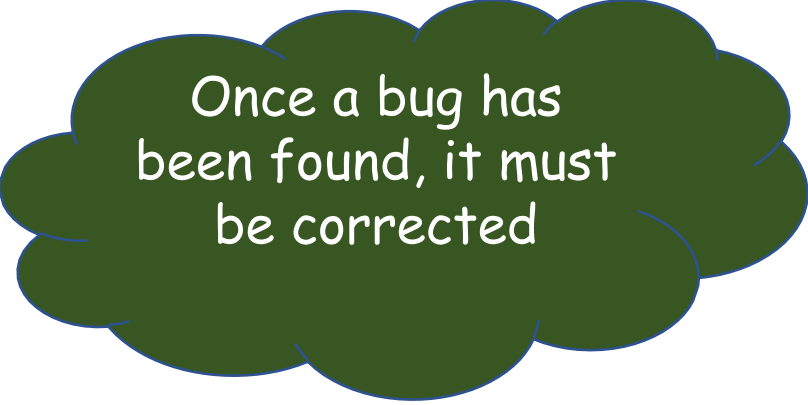
- Backtracking is a fairly common debugging approach

induction

- Induction or deduction introduces the concept of **binary partitioning**

deduction

Correcting the Error



Once a bug has been found, it must be corrected

Three simple questions that you should ask *before making* the "*correction*" that removes the cause of a bug.

*Is the cause of the **bug reproduced in another** part of the program?*

*What "**next bug**" might be introduced by the **fix I'm about to make**?*

*What could we have **done to prevent this bug in the first place**?*

hello world